The Journal of Finance and Data Science 11 (2025) 100177

Contents lists available at ScienceDirect

The Journal of Finance and Data Science

journal homepage: www.keaipublishing.com/en/journals/jfds



Research Article

Pairs trading with time-series deep learning models

Selin Yilmaz [a], Emre Sefer [b,*]

[a] Computer Science Department, Nisantepe, Orman Sk. No:13, 34794, Istanbul, Turkey
[b] Ozyegin University Artificial Intelligence and Data Engineering Department, Nisantepe, Orman Sk. No:13, 34794, Istanbul, Turkey

ARTICLE INFO

Keywords:
Pairs trading
Statistical arbitrage
Deep learning
Transformer
Autoformer
Informer
iTransformer
Scaleformer
Chronos

ABSTRACT

Pairs trading is a well-studied statistical arbitrage strategy including the identification of asset pairs exhibiting correlated changes in their historical prices. This statistical arbitrage strategy focuses on benefiting from non-permanent divergent behaviour of price, and it forecasts that the price relationship will revert to its usual and normal correlation. In this paper, we explore how more recent time-series-based deep learning techniques can be utilized in pairs trading, where cointegrated asset pairs are taken into account. We propose deep-learning and more traditional machine learning-based methods to predict the fluctuation of daily idiosyncratic residual terms between assets and their factor approximations. In our analysis, we focused on seven models: LSTM as a fundamental time-series method to capture interrelationships in dataset, Informer, Autoformer, iTransformer, Scaleformer, and Chronos as transformer-based deep time series methods, and AdaBoost, which is an ensemble learning-based machine learning method. We have assessed the performance of methods comprehensively over S&P 500 and cryptocurrency assets data starting from 2012 to 2020 respectively, and used a traditional statistical arbitrage-based relative value method as a baseline. All of our proposed learning-based methods turned out to be profitable strategies, obtaining higher Sharpe ratios and average returns by outperforming the baseline relative value method. Nevertheless, deep learning-based methods had a lower volume than the baseline, so when transaction costs are taken into account they showed better performance. Deep learning-based methods maximum drawdown was also lower than the traditional statistical arbitrage strategy. As a result, we show the benefits of time series-based deep learning methods in pairs trading across distinct asset classes.

1. Introduction

Statistical arbitrage is a well-studied quantitative trading strategy, and hedge funds have focused on it since its invention in the 1980s (Krauss, 2017). Statistical arbitrage term includes a distinct set of investment strategies, focusing on inferring and benefiting from almost identical asset price differences via statistical techniques. These statistical arbitrage strategies notionally depend on comparable and correlative trades between an asset and a mimicking portfolio. In this case, the construction of a mimicking portfolio is done in a way that the portfolio resembles to target asset in terms of historical price comovements. Pairs trading is a well-studied statistical arbitrage strategy that includes the determination of cointegrated asset pairs. The assumption is that the difference between these pair of assets returns is a stationary process. According to this strategy, one takes a long position in one of the assets by

* Corresponding author.
E-mail address: emre.sefer@ozyegin.edu.tr (E. Sefer).
Peer review under the responsibility of KeAi Communications Co., Ltd.

https://doi.org/10.1016/j.jfds.2026.100177
Received 20 July 2025; Received in revised form 15 January 2026; Accepted 28 January 2026
Available online 4 February 2026
2405-9188/© 2026 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

buying the asset as well as takes a short position in the remaining asset by short-selling the asset, especially when the spread between these pair of assets increases above a certain expected value. In our case, the difference in returns between two assets is called the spread, and pairs trading's aim is to benefit when the spread reverses. Even though pairs trading is a well-studied and common strategy, it also has a significant risk of the spread not being reverted. Such risk became more apparent by the situation of the hedge fund Long-Term Capital Management where 1998 Asian criss resulted in the fund to lose around 4 billion via pairs trading and shutting down its operations (Kho et al., 2000).

Generalized pairs trading is a novel augmentation to the traditional pairs trading discussed above, where it uses factors for return decomposition in factor model. Via generalized pairs trading, cointegration pair search becomes easier. Moreover, generalized pairs trading utilizes assets portfolio, remarkably decreasing the potential market risk. In terms of decomposition, the factor model is utilized to break down the returns of an asset via a number of factors which are related with asset's return and risk. Practically, there are many alternatives for those factors. Three of these alternatives are: 1- Those factors can be estimated via firms focusing on factor models such as MSCI-BARRA (Jorge et al., 2019; Kho et al., 2000)- Those factors could be obtained by running Principal Component Analysis (PCA), and 3- Market ETFs could be used for these factors.

In this paper, we focus on investigating six time series-based deep learning methods and one machine learning approach to model stationary residuals. While evaluating the effectiveness of our approaches, our baseline is a well-studied statistical arbitrage technique called the relative value approach. Among these three deep learning techniques, Long Short-Term Memory (LSTM) (Hochreiter and Schmidhuber, 1997) is a time series-based approach focusing on capturing the longer-term interrelationships over the associated time series dataset. As the training continues, a greater amount of train data could be incorporated via LSTM. On the other hand, our baseline relative value model needs intervals of predetermined length while statistical parameters are being estimated and compared. Such a predetermined length of intervals is one of the limitations of the relative value model. Additionally, Long Short-term Memory is designed to keep the knowledge from far off history, as long as such distant data is important in explaining the current data. Another time series-based deep learning approaches are Informer (Zhou et al., 2021), Autoformer (Wu et al., 2022), iTransformer (Liu et al., 2023), Scaleformer (Shabani et al., 2022), and Chronos (Ansari et al., 2024) where all of these are based on transformers (Vaswani et al., 2023).

Informer (Zhou et al., 2021) is a variant of the transformer model, specifically designed for efficient long-term time-series forecasting. It addresses the limitations of standard transformers in terms of computational complexity and memory usage, especially for long sequences of time-series data. On the other hand, Autoformer (Wu et al., 2022) is a transformer-based model designed for time-series forecasting, with a focus on improving efficiency and accuracy in capturing long-term temporal patterns. It integrates series decomposition into account while focusing on autocorrelations. It introduces novel mechanisms to enhance the performance of standard transformers on time-series data by addressing issues like over-smoothing and poor long-term trend capture. Among the other time-series-based transformer techniques, iTransformer (Liu et al., 2023) is a unified transformer-based model introducing an instance-based learning paradigm, where time series are processed at the instance level rather than the traditional variable level. This enables the model to generalize across datasets and domains by treating each time series instance independently. Scaleformer (Shabani et al., 2022) introduces a scale-invariant design to better capture temporal patterns across multiple time scales. Unlike traditional transformers that use fixed positional encoding and uniform attention, Scaleformer employs a pyramid attention mechanism and multi-resolution patching, which allows it to process time series data at different temporal resolutions. Ansari et al. (2024) unifies multiple model architectures under a shared encoder-decoder design, enabling zero-shot, few-shot, and supervised learning within a single system. It reformulates time series forecasting as a sequence-to-sequence problem, leveraging a shared pre-trained encoder to extract temporal features and task-specific decoders to adapt to different downstream forecasting tasks. Our more traditional machine learning-based technique is AdaBoost (Schapire, 1999), which is an ensemble-based method focusing on boosting. Even though autocorrelations in time-series data is not modelled by AdaBoost, ensemble learning-based AdaBoost has shown robust and effective performance when it focuses on modeling time-series data (Al-Hadeethi et al., 2020; Chang et al., 2019; Chuan et al., 2021; Ribeiro and dos Santos Coelho, 2020). In our case, we will analyze AdaBoost's efficiency complement to deep learning-based time series techniques such as LSTM, Informer, Autoformer, iTransformer, Scaleformer, and Chronos.

All techniques except the baseline relative value model focus on extracting a general pattern between assets by analyzing a set of residuals, however the relative value model focuses only on one asset. All techniques except the baseline relative value model are based on analyzing a panel of residuals. In the panel approach, residual movements are explained via the whole cross-section's historical changes. Panel approach can be seen as a predictive framework where asset contributions are removed from the noisy part due to the reciprocal elimination of random errors. In contrast, the relative value model is almost entirely statistical, which is more prone to fetching noise.

This study advances the statistical arbitrage and pairs trading literature by reframing generalized pairs trading as a panel-level residual direction prediction problem, rather than a collection of asset-specific mean-reversion estimations. Unlike classical relative-value approaches that rely on parametric assumptions and fixed rolling windows, and unlike prior learning-based studies that focus on single-model adoption, we develop a unified framework in which modern time-series models are trained jointly across the cross-section of factor-residuals. This formulation enables the learning of shared temporal dynamics, long-range dependencies, and regime-dependent patterns that are not captured by traditional Ornstein–Uhlenbeck or asset-wise predictive models. Methodologically, we provide the first comprehensive evaluation of recent time-series transformers, including Informer, Autoformer, iTransformer, Scaleformer, and Chronos, within a realistic statistical arbitrage setting, alongside LSTM and ensemble-based learning baselines. Empirically, we introduce a survivorship-bias-aware backtesting protocol with transaction costs across both equity and cryptocurrency markets, allowing a controlled comparison between parametric statistical methods and prediction-driven models. Our results demonstrate that the benefits of deep time-series models in pairs trading arise not merely from architectural complexity, but from their

ability to capture cross-asset temporal structure and deliver superior risk-adjusted performance with lower trading turnover, thereby positioning prediction-based residual modeling as a viable and practically relevant alternative to classical mean-reversion strategies.

In our analysis, we focused on S&P 500 index's asset and cryptocurrency assets data starting from 2012 to 2020, respectively. In order to prevent survivorship bias, index constituent assets are tracked uninterruptedly for each day. During the period starting from 2012 for S&P 500 dataset, we trained and tested the methods, only focusing on active assets belonging to the index. Overall, our contributions can be summarized as follows:

- To the best of our knowledge, for the first time, we comprehensively discuss the performance of deep time series transformers on pairs trading. The performance between different pairs trading algorithms differs, but generally, they outperform the baselines.
- We systematically analyze pairs trading in cryptocurrencies, which has previously been studied in a limited number of studies with more traditional pairs trading techniques.
- We discuss the performance in terms of both machine learning evaluation metrics and portfolio-based measures.

We organize the paper as follows: A brief survey of related research is provided in Section 1.1. Section 2 will give details about our predictive approaches and how we can generate trading strategies over the inferred signals. The details about the experimental dataset and evaluation criteria are described in Section 3. Finally, our results are discussed in Section 4, and the conclusion is discussed in Section 6.

1.1. Related work

1.1.1. Deep learning for financial time-series forecasting

Recent years have witnessed rapid adoption of deep learning techniques in time series forecasting (Kleyman et al., 2017), especially financial time-series forecasting, driven by their ability to model nonlinear dynamics, long-range dependencies, and regime-dependent behavior that are difficult to capture using classical econometric models (Ergun and Sefer, 2025; Erva Ergun and Sefer, 2025; Alaygut and Sefer, 2025a). Early deep learning approaches in finance primarily relied on recurrent neural networks (RNNs) and Long Short-Term Memory (LSTM) architectures, which demonstrated improvements over linear autoregressive and GARCH-type models in tasks such as stock return prediction, volatility forecasting, and market direction classification. These models leverage gated memory mechanisms to retain relevant historical information and have been widely applied to equity, foreign exchange, and cryptocurrency markets.

Subsequent research extended LSTM-based frameworks by incorporating convolutional layers, attention mechanisms, and hybrid architectures to improve feature extraction and temporal alignment (Gezici and Sefer, 2025; Alaygut and Sefer, 2025b). CNN-LSTM and attention-augmented RNN models have been shown to enhance predictive accuracy, particularly in noisy financial environments where local temporal patterns and abrupt regime shifts coexist. However, many of these approaches remain sensitive to input window length, require extensive hyperparameter tuning, and are often trained in an asset-specific manner, limiting their ability to generalize across assets or market regimes (Uygun and Sefer, 2025; Celik and Sefer, 2025; Sefer, 2025).

Transformer-based architectures have emerged as a powerful alternative for time-series modeling, offering flexible attention mechanisms that explicitly model long-range temporal dependencies without relying on recurrent state propagation (Tuncer et al., 2022). Originally developed for natural language processing, Transformers have been adapted for time-series forecasting through architectural modifications that address temporal ordering, seasonal structure, and computational efficiency. Notable examples include Informer (Zhou et al., 2021), Autoformer (Wu et al., 2022), iTransformer (Liu et al., 2023), Scaleformer (Shabani et al., 2022), and Chronos (Ansari et al., 2024), which introduce innovations such as sparse attention, series decomposition, instance-wise modeling, multi-scale representations, and pretraining-based temporal encoders.

In financial applications, Transformer-based models have demonstrated strong performance in forecasting asset prices, volatility, and macroeconomic indicators, particularly when long historical contexts or cross-asset dependencies are present. Their ability to reweight temporal information dynamically makes them well-suited for non-stationary financial data, where the relevance of past observations varies across market regimes. Nevertheless, much of the existing literature evaluates these models in isolation, focuses on point forecasting accuracy, or neglects realistic trading constraints such as transaction costs, turnover, and survivorship bias.

1.1.2. Pairs trading

In finance, pairs trading, generalized pairs trading, and its other variants are widely recognized and widely adopted by the industry since the expected profit from such a statistical arbitrage strategy is higher than many other strategies as well as these strategies are not significantly correlated with distinct types of investment approaches. These pairs trading strategies have become more popular over time since they are effective in exploiting inefficient markets due to pricing differences, so these strategies are being researched more frequently as one of the potential investment strategies.

The existing research on pairs trading and variants have focused on identifying securities that move together, which are time series-based methods such as autoregressive ARCH or GARCH methods which infer temporally-changing interdependencies occurring in the price (Chen et al., 2014; Cummins and Bucca, 2012; Zeng and Lee, 2014), cointegration test-based approaches (Lin et al., 2006; Hossein Rad and Faff, 2016; Vidyamurthy, 2004), and distance-based approaches which measure the dependence between securities in terms of the distance between two time-series (Do and Faff, 2010; Gatev et al., 2006). Recently, machine learning methods have become more and more popular, mainly due to major computational power increases in hardware and artificial intelligence-associated developments. As an example, Sarmento and Horta (2020) has come up with an unsupervised learning-based method to infer a

candidate pair of securities via unsupervised learning and then forecast trading behaviour in order to reduce the portfolio's maximum drawdown duration. In another work, Brim (2020) proposed a reinforcement learning-based method which is inspired by Double Deep Q-Network (DDQN) (Van Hasselt et al., 2016) to learn pairs of securities. According to their detailed experiments, their DDQN-based approach successfully identifies cointegrated pair of securities mean reversion characteristics which results in precise estimations depending on this mean reversion characteristics.

There are a number of recent studies focusing on pairs trading with cryptocurrency assets, which refine pairs trading strategies tailored to the unique dynamics of cryptocurrency markets. Tadi and Witzany (2025) proposes a copula-based strategy applied to cointegrated crypto pairs, using both linear and nonlinear tests to identify trading candidates and generate mispricing signals. Their high-frequency backtests (based on 5-min data) demonstrate exceptional results, with annualized net returns reaching up to 205.9 %, Sharpe ratios as high as 3.77. Another frontier explored is the integration of machine learning techniques—especially reinforcement learning (RL)—into pairs trading. Yang and Malik (2024) introduces a dynamic RL-based framework applied to BTC-GBP and BTC-EUR 1-min data sets. Compared to a traditional non-RL strategy yielding annualized returns of around 8.33 %, their RL agent achieved returns in the range of 9.94 % to 31.53 %, depending on training configurations and reward shaping. This research illustrates that adaptive decision-making through RL can meaningfully enhance profitability in volatile crypto markets by dynamically scaling trade decisions. Moreover (Fil and Kristoufek, 2020), employs both distance-based and cointegration-based pairs trading techniques on a selection of 26 highly liquid cryptocurrencies listed on Binance. These strategies are tested across three different time intervals: 5-min, hourly, and daily. Collectively, the literature emphasizes the importance of high-frequency data, careful methodology selection, and advanced modeling—potentially including machine learning—to maintain an edge in pairs trading across cryptocurrencies. Lastly, the review in Ko et al. (2024) investigates the effectiveness of six statistical pair-selection methods, Cointegration, Correlation, Distance, Fluctuation Behaviour, Hurst Exponent, and Stochastic Differential Residual (SDR), applied to intraday price data of 30 cryptocurrencies on Binance. They evaluate each method using 11 performance metrics covering returns, risk, and trade quality.

Among other works, Zhao et al. (2022) is the most similar study to our research in this paper. In their paper, LSTM and Relative Value approaches have been evaluated in terms of their statistical arbitrage profits when generalized pairs trading is utilized. They identified LSTM to perform better than the Relative Value approach, especially if we take the transaction costs into account. When transaction costs are not considered or transactions are assumed to be free, the performance difference between these approaches is lower, but LSTM still outperforms the Relative Value approach. Our work is different than their study in three major ways: 1- We investigate transformer-based time series deep learning methods in depth, such as Informer, Autoformer, iTransformer, Scaleformer, and Chronos, which have recently performed better than LSTM across many distinct tasks (Wang et al., 2023). 2- Our work focuses on handling the survivorship bias better than their approach in S&P 500 index constituents. Their work separated the whole evaluation period into two different periods: between 2001 and 2007, and between 2015 and 2021. However, we follow a daily tracking process for S&P constituents. So, our method is significantly more realistic and can be applied as a real strategy since our evaluation and profit calculations are based on the correct active S&P members at any day. 3- We also compare the performance with an ensemble-based machine learning approach, AdaBoost, which is also effective in time series datasets in finance, as well as being one of the effective algorithmic trading strategies (Chuan et al., 2021).

2. Methods

Here, we will be introducing the eight approaches we use in predicting the daily residuals directional movements of every stock. Afterwards, we will be describing the evaluation criteria and trading strategies in Section 2.10.

2.1. Statistical arbitrage: relative value (RV) approach

We follow the generalized pairs trading strategy by Avellaneda and Lee (2010) where we forecast stock residuals by regressing againts a factor model. In the original pairs trading approach, we regress against another stock. Here, we follow same notations for stock i at time t. Let $R_i(t)$ be the stock return, and we express the return as follows:

$$R_i(t) = \sum_{j=1}^{m} \beta_{ij} F_j(t) + \epsilon_i(t) \tag{1}$$

where factor returns are represented by $F_j(t)$, $j = 1, 2, \ldots, m$ and factor loadings are represented by β_{ij}, $j = 1, 2, \ldots, m$. Residuals after factor decompositions are represented by $\epsilon_i(t)$ which defines a stationary process over time.

According to Equation (1), for each stock, we kind of have asset pair where one asset is represented by R_i and the other one is defined by $\sum_{j=1}^{m} \beta_{ij} F_j(t)$, and the difference between them defines a stationary process $\epsilon_i(t)$. Generalized pairs trading strategy focuses on finding arbitrage and profit occasions on a stock portfolio by modeling the difference ϵ_i's. A well-known approach to model ϵ_i's is via a mean-reverting Orenstein-Uhlenbeck (OU) process (Uhlenbeck and Ornstein, 1930) which has been introduced in Avellaneda and Lee (2010) as part of their relative value (RV) approach. In this case, for each stock constituent of S&P 500, parameters are fitted to the process and they model the reversals of the residuals.

We assume each residual over time ϵ_i follows a continuous-time mean-reverting process as well as the system of stochastic differential equations discussed below are satisfied:

$$\frac{dS_i(t)}{S_i(t)} = \alpha_i dt + \sum_{j=1}^{N} \beta_{ij} \frac{dI_j(t)}{I_j(t)} + dX_i(t) \tag{2}$$

where factor loadings are denoted by β_{ij}'s, $\alpha_i dt$ defines the excess asset return than the market. Additionally, the expression $\sum_{j-1}^{N} \beta_{ij} \frac{dI_j(t)}{I_j(t)}$ is the part of asset return which can be explained by the systematic factors. The remaining term $dX_i(t)$ is the part that is not explained by the factors, and it models a stationary stochastic process in increments. In this Relative Value approach, we model $dX_i(t)$ parametrically by an Ornstein-Uhlenbeck process.

$$dX_i(t) = \kappa_i(m_i - X_i(t))dt + \sigma_i dW_i(t), \quad \kappa > 0 \tag{3}$$

where σ_i, κ_i, m_i, and α_i define parameters specific to assets and we assume their stability over a predefined time period. The remaining term $W_i(t)$ is a Brownian motion. By solving the differential Equation (3), we obtain:

$$X_i(t_0 + \Delta t) = e^{-\kappa \Delta t} X_i(t_0) + (1 - e^{-\kappa_i \Delta t}) m_i + \sigma_i \int_{t_0}^{t_0 + \Delta t} e^{-\kappa_i (t_0 + \Delta t - s)} dW_i(s) \tag{4}$$

where we solve 1 day lag regression model to estimate m_i and κ_i terms with a window of length 60 days. We also have:

$$X_{n+1} = a + bX_n + \zeta_{n+1}, \quad n = 1, 2, \ldots, 59 \tag{5}$$

In Equation (5), we define X_n as:

$$X_n = \sum_{j=1}^{n} \epsilon_n, \quad n = 1, 2, \ldots, 60 \tag{6}$$

which is discretization of $X(t)$ defined in Equation (4). When we combine the Equations (4) and (5), we derive the parameters as follows:

$$a = m(1 - e^{-\kappa \Delta t}) \tag{7}$$

$$b = e^{-\kappa \Delta t} \tag{8}$$

Hence,

$$\kappa = \log(b)^{*}252 \tag{9}$$

$$m = \frac{a}{1 - b} \tag{10}$$

In the Relative Value model, by following [43], 60-day rolling window approach is used and trading signals are defined by the subsequent conditions:

$$\text{buy to open if } s_i < -s_{bo} \tag{11}$$

$$\text{sell to open if } s_i > +s_{so} \tag{12}$$

$$\text{close short position if } s_i < +s_{bc} \tag{13}$$

$$\text{close long position if } s_i > -s_{sc} \tag{14}$$

where s_i the standard deviation of $X_i(t)$ from its average m_i. Trading open/close thresholds are defined as $s_{bo} = 1.25$ $s_{so} = 1.25$, $s_{bc} = 0.75$, and $s_{sc} = 0.5$.

2.2. Adaptive boosting

Adaptive Boosting, commonly known as AdaBoost, is an ensemble learning method that aims to improve the performance of weak classifiers by combining them into a strong classifier. The fundamental idea behind AdaBoost is that rather than building a single complex model, it sequentially trains several simpler models (called weak classifiers) and adjusts their importance based on how well they perform on the data. Its step can be partitioned as follows:

1. **Initial Step:** AdaBoost starts by assigning equal weights to each data point in the dataset. These weights determine the importance of each data point for the training process.
2. **Training Weak Classifiers:** In each iteration or round, a weak classifier is trained using the weighted dataset. A weak classifier is typically a simple model, such as a decision stump (a tree with only one split). After training, the weak classifier is evaluated based

on how well it classifies the data. If it correctly classifies most points, it is considered to have a low error otherwise, it has a high error.

3. **Adjusting Weights:** AdaBoost then adjusts the weights of the data points. Points that were incorrectly classified are given higher weights, making them more important in the next round of training. Points that were classified correctly have their weights reduced. This weight adjustment ensures that in the next round, the weak classifier focuses more on the difficult examples that were misclassified in the previous round.
4. **Classifier Weighting:** Each weak classifier is assigned a weight based on its accuracy. A more accurate classifier gets a higher weight, while a less accurate one gets a lower weight. This ensures that the better-performing classifiers have more influence on the final prediction.
5. **Final Strong Classifier:** After several iterations, AdaBoost combines all the weak classifiers into a single strong classifier. The final prediction is made by taking a weighted vote of all the weak classifiers' predictions. The classifiers with higher weights (i.e., the more accurate ones) have more influence on the final outcome.

Key characteristics of AdaBoost can be summarized as follows: 1- Focuses on difficult data points: By adjusting the weights, AdaBoost forces subsequent classifiers to focus on the points that previous classifiers misclassified. 2- Combines weak classifiers: Each individual weak classifier may not perform well on its own, but when combined, they form a much more accurate model. 3- Versatile: AdaBoost can work with different types of weak classifiers and is not restricted to decision trees. 4- Boosting method: It belongs to the class of boosting algorithms, where the goal is to "boost" the performance of simple models by learning from their mistakes. Here, we used decision trees as the base learners. We used grid search to tune the hyperparameters, where 50 estimators and 0.3 learning rate were found to be the parameters leading to the best performance.

2.3. LSTM

Long Short-Term Memory (LSTM) (Hochreiter and Schmidhuber, 1997) models sequential data, and it is a special case of Recurrent Neural Network (RNN). Contrary to more conventional feedforward neural networks, LSTMs extract temporal knowledge over a given sequence by maintaining internal state. By such maintenance, LSTMs are successful at handling time series datasets where preceding input affects the next output. Some examples of datasets which could be modelled via LSTMs are weather data over time, speech signal, or stock prices, in our case. LSTMs are different than RNNs in the way they utilize 3 specialized types of gates such as output gate, input gate, and forget gate. LSTMs use these gates to keep or forget knowledge over long time horizons. By selectively keeping or forgetting knowledge, LSTMs solve the vanishing gradients problem, which appears in conventional RNNs. An LSTM unit consists of several key components: the cell state, hidden state, and three gates (forget gate, input gate, and output gate).

1. Cell State (C_t): The cell state is responsible for carrying information across the sequence. It acts like a memory, allowing information to flow through the network with minimal modifications. The cell state can maintain information for long durations, and the gates decide what information should be added or removed.
2. Hidden State (h_t): The hidden state is used to capture information about the current time step and is passed to the next time step along with the cell state. It is the "output" of the LSTM unit at each time step.
3. Gates: LSTM networks use gates to control how much information should be kept or discarded. These gates are controlled by learned parameters and consist of sigmoid activation functions (output values between 0 and 1) that decide the extent of the information flow. The key gates are:
   (a) Forget Gate(f_t): Decides what information to discard from the cell state. It takes the previous hidden state (h_{t-1}) and the current input (x_t) and outputs a number between 0 and 1 for each element in the cell state. A value of 0 means "completely forget," and 1 means "completely retain."

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{15}$$

   (b) Input Gate (i_t): Controls how much new information from the current input should be added to the cell state. It works in conjunction with the input modulation gate to update the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{16}$$

   (c) Output Gate (o_t): Determines what part of the cell state should be output as the hidden state for the next time step. This output is filtered through a tanh function to scale the values between -1 and 1.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{17}$$

Practically, LSTMs obtained encouraging and favourable results across a distinct set of applications such as extracting the composition of music (Chaki et al., 2020; Chuan and Herremans, 2018), making predictions over time series datasets (Cao et al., 2019; Laptev et al., 2017; Sagheer and Kotb, 2019), recognizing speech (Graves et al., 2013; Jorge et al., 2019), and much more. Our LSTM

architecture is similar to the one discussed in Zhao et al. (2022). LSTM model's input is 60 days of historical idiosyncratic returns and the model forecasts the next residual term's direction relative to the current residual.

2.4. Informer

Informer (Zhou et al., 2021) is a variant of the transformer model (Vaswani et al., 2023), specifically designed for efficient long-term time-series forecasting. It addresses the limitations of standard transformers in terms of computational complexity and memory usage, especially for long sequences of time-series data. Below is a concise mathematical summary of the Informer model, focusing on its key components, including the ProbSparse attention mechanism, sequence distillation, and its generative forecasting strategy. For a time-series forecasting task, the input sequence is denoted as $X = \{x_1, x_2, \ldots, x_T\}$, where $x_t \in \mathbb{R}^d$ is a d-dimensional vector at time t. The goal is to predict the future values $\widehat{X} = \{\widehat{x}_{T+1}, \widehat{x}_{T+2}, \ldots, \widehat{x}_{T+k}\}$ over the forecasting horizon k.

1. **ProbSparse Attention:** Traditional self-attention has a time complexity of $O(L^2)$, where L is the length of the sequence, due to the need to compute pairwise dot products for all queries and keys. Informer introduces ProbSparse Attention to reduce this complexity by focusing only on the most important queries. The idea is to sparsify the attention matrix by only selecting a subset of queries that carry the most information. In standard self-attention, the output for a query q_i is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{18}$$

where $Q \in \mathbb{R}^{L\times d_k}$ are the queries, $K \in \mathbb{R}^{L\times d_k}$ are the keys, $V \in \mathbb{R}^{L\times d_v}$ are the values, and d_k is the dimensionality of queries and keys. To reduce computational complexity, ProbSparse Attention approximates the attention matrix by focusing on the top-u queries with the largest sparsity scores $P(q_i)$, defined as:

$$P(q_i) = \max_{k\in K}\left(\frac{q_i k_j^T}{\sqrt{d_k}}\right) \tag{19}$$

Then, the attention is computed only for the top-u queries with the highest scores. This reduces the attention complexity from $O(L^2)$ to $O(L \log\ L)$.

2. **Sequence Distillation:** Informer further reduces the computational complexity by compressing the input sequence length using a **distillation operation**. The idea is to aggregate information across longer spans of time to form a compressed representation of the input sequence, reducing the length from L to L/r, where r is the distillation ratio. The distillation process is mathematically defined as:

$$X_d = \text{Distill}(X) \tag{20}$$

where $X_d \in \mathbb{R}^{(L/r)\times d}$ is the distilled sequence, and the Distill function compresses the sequence by averaging or pooling over non-overlapping windows of size r. This operation reduces the length of the sequence, making the model more efficient without losing essential temporal information.

3. **Generative Forecasting:** Informer uses a generative forecasting approach, where future time steps are predicted sequentially. At each step, the previously predicted value is fed back into the model to generate the next prediction. The generative process is formulated as:

$$\widehat{x}_{T+1} = f(X, \widehat{x}_T) \tag{21}$$

$$\widehat{x}_{T+2} = f(X, \widehat{x}_T, \widehat{x}_{T+1}) \tag{22}$$

$$\widehat{x}_{T+k} = f(X, \widehat{x}_T, \ldots, \widehat{x}_{T+k-1}) \tag{23}$$

where f represents the forecasting function learned by the Informer. Each future value $\widehat{x}_{T+k}$ is predicted using both the past observed values and previously generated future values.

4. **Encoder-Decoder Architecture:** Informer employs the encoder-decoder architecture similar to the original transformer model. The encoder takes the input sequence X and applies the ProbSparse Attention mechanism and sequence distillation to generate a compact representation:

$$H_e = \text{Encoder}(X_d) \tag{24}$$

where H_e is the encoded representation of the distilled input sequence X_d. The decoder then takes this encoded representation and iteratively generates the future time steps using the generative forecasting strategy:

$$\widehat{X} = \text{Decoder}(H_e, \widehat{X}_{T+1}, \widehat{X}_{T+2}, \dots) \tag{25}$$

2.5. Autoformer

Autoformer (Wu et al., 2022) is a transformer-based model (Vaswani et al., 2023) designed for time-series forecasting, with a focus on improving efficiency and accuracy in capturing long-term temporal patterns. It introduces novel mechanisms to enhance the performance of standard transformers on time-series data by addressing issues like over-smoothing and poor long-term trend capture. Here's a summary of the Autoformer model, focusing on its core components and mechanisms.

1. **Series Decomposition:** Autoformer explicitly decomposes the time-series data $X = \{x_1, x_2, \dots, x_T\}$ into trend and seasonal components. The decomposition can be expressed as:

$$X = T + S \tag{26}$$

where T is the trend component, usually smoothed over time, and S is the seasonal component, capturing high-frequency fluctuations. Trend component captures the long-term, slow-changing part of the data whereas the seasonal component models short-term fluctuations and repeating patterns.

2. **Decomposition Block:** In Autoformer, the input time-series is decomposed into the trend and seasonal components using a moving average or similar smoothing filter:

$$S_t = X_t - \text{MA}(X_t) \tag{27}$$

$$T_t = \text{MA}(X_t) \tag{28}$$

where S_t is the seasonal component at time t, T_t is the trend component at time t, and $\text{MA}(X_t)$ represents a moving average of the time series, smoothing out high-frequency changes.

3. **Auto-Correlation Mechanism:** Autoformer replaces the traditional self-attention mechanism with the Auto-Correlation mechanism, which focuses on periodic patterns in the time series. The goal is to capture dependencies between points that are distant but exhibit repeating behavior. The Auto-Correlation function is mathematically defined as:

$$\text{ACF}(t, \tau) = \frac{\sum_{i=1}^{n-\tau} (X_t[i] \cdot X_t[i+\tau])}{n - \tau} \tag{29}$$

where $X_t[i]$ is the value of the time series at time i, τ is the lag or shift in time, and $\text{ACF}(t, \tau)$ measures the correlation between time points separated by τ. The Auto-Correlation mechanism selects and strengthens the most relevant correlations, reducing the number of time points the model needs to attend to.

4. **Encoder-Decoder Architecture:** Autoformer follows the traditional encoder-decoder architecture. The encoder takes the input time-series X and applies a series of decomposition blocks:

$$\text{Encoder}(X) = \{T, S\} = \text{Decompose}(X) \tag{30}$$

where the input series X is decomposed into trend T and seasonal S components. The seasonal component is processed through the Auto-Correlation mechanism to capture key dependencies, while the trend component is smoothed. The decoder progressively reconstructs the future series from the processed seasonal and trend components. Using progressive decoding, the decoder generates future time steps iteratively:

$$\widehat{X}_{t+k} = \text{Decoder}(T, S, \widehat{X}_{t+1}, \widehat{X}_{t+2}, \dots, \widehat{X}_{t+k-1}) \tag{31}$$

where $\widehat{X}_{t+k}$ is the predicted value at time $t + k$, and each step of prediction depends on previous predictions $\widehat{X}_{t+1}, \widehat{X}_{t+2}, \dots$.

5. **Progressive Refinement:** Autoformer refines its predictions by iterating over the trend and seasonal components, progressively improving its forecasts as more future time steps are predicted:

$$\widehat{X}_{t+k} = T_{t+k} + S_{t+k} \tag{32}$$

where $\widehat{X}_{t+k}$ is the predicted value for the time step $t + k$, reconstructed from both the trend T_{t+k} and seasonal S_{t+k} components.

2.6. iTransformer

iTransformer (Liu et al., 2023) is a unified Transformer-based model designed for time series forecasting, addressing both long-term and short-term prediction tasks. It introduces an instance-based learning paradigm, where time series are processed at the instance

level rather than the traditional variable level. This enables the model to generalize across datasets and domains by treating each time series instance independently. iTransformer leverages a patch embedding strategy to convert time series into fixed-length representations, followed by standard Transformer blocks to model temporal dependencies. Its instance-centric design allows for both effective pretraining and transfer learning, outperforming many existing methods in forecasting benchmarks. Its key contributions are: 1- Instance Normalization tailored for time series, 2- Autocorrelation-based self-attention (like Informer), 3- Seasonal-trend decomposition pre-processing, and 4- Encoder-decoder Transformer architecture adapted for time series.

Let a multivariate time series be denoted as:

$$X = \{x_1, x_2, \ldots, x_T\} \in \mathbb{R}^{T \times d} \tag{33}$$

where T is the sequence length and d is the feature dimension. The model applies a linear embedding layer:

$$E = \text{Linear}(X) \in \mathbb{R}^{T \times d}\text{model} \tag{34}$$

and adds positional encodings $P \in \mathbb{R}^{T \times d_{\text{model}}}$, $H_0 = E + P$. Each instance is normalized individually across the time dimension:

$$\mu^{(i)} = \frac{1}{T}\sum_{t=1}^{T} X_t^{(i)}, \quad \sigma^{(i)} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(X_t^{(i)} - \mu^{(i)}\right)^2 + \epsilon} \tag{35}$$

$$\text{IN}(X_t^{(i)}) = \frac{X_t^{(i)} - \mu^{(i)}}{\sigma^{(i)}} \tag{36}$$

This ensures each input sequence has zero mean and unit variance, facilitating better generalization. Instead of standard self-attention, iTransformer uses autocorrelation-based attention inspired by the Informer model. The model computes correlation using the Fast Fourier Transform (FFT):

$$\text{Corr}_\tau(Q, K) = \mathcal{F}^{-1}(\mathcal{F}(Q) \cdot \overline{\mathcal{F}(K)}) \tag{37}$$

where $\mathcal{F}$ denotes the Fourier transform, $\cdot$ is element-wise multiplication, and $\overline{\mathcal{F}(K)}$ is the complex conjugate of $\mathcal{F}(K)$. The model selects the top-k lags with the highest correlation:

$$\text{TopK}_\tau = \text{argmax}_k \text{Corr}_\tau(Q, K) \tag{38}$$

Then it computes attention as following, which allows the model to capture periodic dependencies efficiently:

$$\text{Attention}(Q, K, V) = \sum_{j=1}^{k} \alpha_j \cdot \text{shift}(V, \tau_j) \tag{39}$$

Given the encoder output H_{enc} and decoder input H_{dec}, the forecast for the next L steps is:

$$\widehat{Y}_{T+1:T+L} = \text{Decoder}(H_{\text{enc}}, H_{\text{dec}}) \tag{40}$$

2.7. Scaleformer

Scaleformer (Shabani et al., 2022) is a transformer-based time series forecasting model designed specifically for long-term forecasting. It addresses challenges in modeling multiscale temporal dependencies by introducing a novel scale-aware hierarchical representation learning approach. Time series often exhibit temporal patterns across different scales, such as short-term fluctuations, long-term trends, and seasonal cycles. Vanilla Transformers, which apply attention uniformly, struggle to effectively model such multiscale temporal dependencies. Scaleformer addresses this limitation via a hierarchical, scale-aware design.

The input sequence is progressively downsampled to create a temporal hierarchy:

$$X^{(0)} = X, \quad X^{(1)} = \text{Downsample}(X^{(0)}), \quad X^{(2)} = \text{Downsample}(X^{(1)}), \ldots \tag{41}$$

Let $X^{(s)} \in \mathbb{R}^{T_s \times d}$ be the sequence at scale level s, where $T_s < T_{s-1}$. Downsampling can be performed using average pooling or convolution with a stride. Each scale s is processed using an independent Transformer encoder:

$$H^{(s)} = \mathcal{E}^{(s)}(X^{(s)}) \in \mathbb{R}^{T_s \times d} \tag{42}$$

Each encoder consists of: 1- Multi-head self-attention (MHSA), 2- Feed-forward network (FFN), and 3- Residual connections and LayerNorm. To enable interaction between different temporal resolutions, Scaleformer uses cross-scale attention. A fine-scale query attends to a coarse-scale key/value pair:

$$Q = H^{(s)}, \quad K = H^{(s+1)}, \quad V = H^{(s+1)} \tag{43}$$

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \tag{44}$$

The fused representation is then:

$$\widetilde{H}^{(s)} = H^{(s)} + \text{Attention}(H^{(s)}, H^{(s+1)}, H^{(s+1)}) \tag{45}$$

This cross-scale mechanism is applied in a top-down manner (from coarse to fine). The final multi-scale representations are upsampled to match the original temporal resolution:

$$\widehat{H} = \text{Upsample}(\widetilde{H}^{(1)}) + \cdots + \text{Upsample}(\widetilde{H}^{(S)}) \tag{46}$$

A linear projection layer maps the fused features to the output space:

$$\widehat{Y} = \text{Linear}(\widehat{H}_{[-L:]}) \tag{47}$$

where $\widehat{H}_{[-L:]}$ denotes the final L time steps of $\widehat{H}$.

2.8. Chronos

Chronos model (Ansari et al., 2024) is a unified time series forecasting framework that leverages pretraining and fine-tuning principles from NLP and computer vision to learn robust temporal representations. It is specifically designed to work across multiple datasets and tasks, with the goal of achieving state-of-the-art performance in both zero-shot and fine-tuned forecasting settings.

Chronos consists of: 1- A shared backbone encoder f_θ (e.g., Transformer), 2- A task-specific head h_ϕ, 3- A self-supervised pretraining phase, and 4- A fine-tuning phase for downstream tasks. Inspired by BERT (Devlin et al., 2019), Chronos masks random time steps and reconstructs them. Let $M \subset \{1, \ldots, T\}$ be masked positions. The model learns to predict:

$$\widehat{x}_t = f_\theta(X_{\setminus M}), \quad \forall t \in M \tag{48}$$

The loss is the mean squared error (MSE):

$$\mathcal{L}_{\text{MTM}} = \frac{1}{|M|}\sum_{t\in M}\|x_t - \widehat{x}_t\|^2 \tag{49}$$

Chronos also applies Contrastive Predictive Coding (CPC). Chronos also learns to distinguish future true embeddings from negative samples. Let $z_t = f_\theta(x_{1:t})$ be the context embedding, and x_{t+k} a future time step. The InfoNCE loss is:

$$\mathcal{L}_{\text{CPC}} = -\sum_{k=1}^{K}\log\frac{\exp(\text{sim}(z_t, x^+_{t+k}))}{\sum_{x^-\in\mathcal{N}}\exp(\text{sim}(z_t, x^-))} \tag{50}$$

where $\text{sim}(a,b) = \frac{a^\top b}{\|a\|\|b\|}$ is cosine similarity, x^+_{t+k} is a positive sample, and $\mathcal{N}$ is a set of negatives. Chronos also aligns signals in the frequency domain. Let $\mathcal{F}(x)$ denote the Fourier transform of x, then the loss is:

$$\mathcal{L}_{\text{Fourier}} = \|\mathcal{F}(x) - \mathcal{F}(\widehat{x})\|^2 \tag{51}$$

which helps preserve periodic structures in the signal. Chronos is fine-tuned on labeled forecasting datasets $\{(X^{(i)}, Y^{(i)})\}_{i=1}^N$. Chronos uses a transformer-based encoder, with the following structure:

$$Z_0 = X + P \quad (\text{input} + \text{positional embedding}) \tag{52}$$

$$Z_\ell = \text{MHSA}(\text{LN}(Z_{\ell-1})) + Z_{\ell-1} \tag{53}$$

$$Z'_\ell = \text{FFN}(\text{LN}(Z_\ell)) + Z_\ell \tag{54}$$

where MHSA is Multi-Head Self-Attention, FFN is Feed-Forward Network, and LN represents Layer Normalization. Chronos has zero-shot and fine-tuned inferences. In Zero-shot inference, Chronos can be used directly on new tasks without retraining. In fine-tuned inference, Chronos can adapt to specific tasks with a small labeled set.

2.9. Architectures comparison

Table 1 highlights the conceptual and methodological differences between classical relative-value pairs trading, prior LSTM-based approaches, and the proposed framework. The key contribution of this study lies not in introducing a new neural architecture, but in reformulating statistical arbitrage as a panel-level residual prediction problem and designing a learning and evaluation pipeline consistent with this perspective.

Classical relative-value methods model residual dynamics using parametric mean-reversion processes estimated independently for each asset over fixed rolling windows. While analytically convenient, these approaches rely on strong stationarity assumptions and ignore cross-asset information, limiting their adaptability to regime changes and increasing sensitivity to noise.

Prior learning-based extensions relax parametric assumptions by learning residual dynamics directly from data, most commonly using LSTM models. However, these methods typically retain an asset-wise modeling paradigm, training separate models per asset or pair. As a result, temporal dependencies are learned in isolation, and cross-sectional structure is not explicitly exploited, which can hinder generalization and increase overfitting risk.

In contrast, the proposed framework models residuals jointly across the asset cross-section, enabling the learning of shared temporal patterns, cross-asset synchronization, and regime-dependent dynamics. The framework is architecture-agnostic and supports multiple modern time-series models, allowing a systematic comparison of temporal inductive biases under a unified trading protocol.

Finally, Table 1 underscores differences in evaluation realism. By explicitly incorporating transaction costs, turnover, and survivorship-bias-aware backtesting, the proposed framework offers a more faithful assessment of deployable statistical arbitrage strategies. Empirically, the results show that prediction-driven panel models achieve superior risk-adjusted performance with fewer trades, indicating that their advantage stems from improved residual modeling rather than increased trading activity.

2.10. Trading strategies

For the baseline relative value model, we apply the identical trading procedure discussed in Avellaneda and Lee (2010). In terms of the remaining 7 methods, we trained each method to predict the succeeding residual term's direction. While making predictions, the probability p is generated by all those methods for every possible stock in the index. Then, we utilized the 0.8 optimum threshold value, which is discussed in Zhao et al. (2022). Particularly, we long a stock when $p > 0.5$ and we close a long position when $p < 0.2$. Similarly, we short a stock when $p < 0.5$ and close the short position when $p > 0.8$. We reconstruct portfolios every day, and our trades goal is to be as close as possible to the residuals where residuals are the mismatch between factor approximation and asset. Our goal with those trades is to capitalize on the trade signals inferred from the dataset.

3. Experiments

3.1. Dataset

The study utilizes two distinct datasets to evaluate the efficacy of pairs trading in varying market conditions: S&P 500 index constituents and cryptocurrency datasets. We obtained the price time series of both datasets daily prices by using the yfinance library from Yahoo Finance (Enkhsaikhan, 2023). S&P 500 index dataset is composed of almost 740 stocks, which was part of the index between January 2010 and to the end of December 2024. We have mitigated survivorship bias by tracking S&P 500 index constituents each possible day. So, when we train and test the models, we use actively trading tickers as part of the index. On the other hand, the cryptocurrency dataset comprises daily closing prices from January 2020 to the end of December 2024 for 20 assets selected from the 50 highest market capitalization cryptocurrencies, based on data availability. This dataset includes FIL-USD, ETH-USD, BTC-USD, and 17 other assets. For cryptocurrency dataset, included assets are shown in Table 2. Compared to cryptocurrencies, the S&P 500 index is characterized by relatively lower volatility. Testing periods are between January 2022 and the end of December 2024 for S&P 500

Table 1
Conceptual and methodological comparison between classical relative-value pairs trading, prior LSTM-based extensions, and the proposed panel-level prediction framework.

Aspect	Classical Relative Value (RV)	Prior LSTM-based Pairs Trading	Proposed Framework (This Study)
Core modeling assumption	Parametric mean reversion (Ornstein-Uhlenbeck process)	Learned mean reversion in single-asset residuals	Prediction-driven residual dynamics without parametric assumptions
Unit of modeling	Individual asset	Individual asset	Cross-sectional panel of assets
Residual modeling	Asset-wise OU parameter estimation	Asset-wise supervised sequence modeling	Joint panel-level residual direction modeling
Temporal dependency handling	Short rolling windows (fixed length)	Limited long-term dependency via recurrence	Long-range dependencies via modern time-series models
Cross-asset information sharing	None	None or implicit	Explicitly learned across assets through shared training
Model adaptability to regimes	Limited (periodic parameter re-estimation)	Moderate	High (data-driven temporal pattern learning)
Architectural scope	Fully parametric statistical model	Single deep architecture (LSTM)	Multiple modern time-series architectures
Evaluation focus	Mean-reversion fit, returns	Returns	Returns, risk, turnover
Key limitation	Strong parametric assumptions, noise sensitivity	Asset-specific overfitting	Higher complexity and data requirements

asset datasets, whereas it is between January 2024 and the end of December 2024 for the cryptocurrency dataset. Tested periods for both asset types consist of both bull and major bear markets.

Since we are applying an evaluation on a time-series dataset, we chose the train and test datasets in continuous intervals. The relative value approach uses 60 days rolling window in recalibrating and finetuning the model parameters and trade signals. We extract the factors associated factor loadings via applying Principal Component Analysis on S&P 500 index assets residual returns by using the foregoing 1 year period for every possible trading window. Contrary to the relative value approach, the remaining methods, including deep learning time-series based methods as well as AdaBoost, predict the up/down direction of the following daily residuals for each asset depending on their existing values. All of those methods take the previous 60 days idiosyncratic returns as input. We train those models by utilizing 3 years of residual returns from all available assets throughout the associated time period. In order to compare these methods fairly with the relative value method, we refine those learning-based models in terms of 60-day intervals, which is aligned with the procedure applied via the relative value approach.

Latent risk factors are extracted using Principal Component Analysis (PCA) applied to the cross-section of asset returns. At each rebalancing date t, PCA is estimated using a rolling one-year (252 trading days) window of standardized daily returns across all actively traded assets. The number of retained principal components is determined by an explained variance threshold of 80 %, which consistently results in retaining the first three principal components (PC1–PC3) across both equity and cryptocurrency datasets. This rolling PCA procedure ensures that factor loadings and factor returns adapt dynamically to changing market conditions while avoiding look-ahead bias.

3.2. Implementation details

All learning-based models are trained using identical rolling-window and refitting procedures to ensure fair comparison with the statistical relative-value baseline. Hyperparameters are selected using a validation period within the training window and then fixed across all test periods. For neural models, we specify optimizer type, learning rate, batch size, number of epochs, early-stopping criteria, and model architecture parameters (e.g., hidden dimensions, number of layers, attention heads). AdaBoost hyperparameters are selected via grid search and held constant thereafter.

The backtesting engine is fully deterministic given a fixed seed and configuration (Zhang et al., 2025a, 2025b). Portfolio construction, trade execution, transaction cost modeling, and performance evaluation are implemented identically across all models. Survivorship-bias-aware constituent tracking is applied consistently throughout the evaluation horizon. In addition to threshold-dependent trading results, threshold-independent metrics (e.g., AUC) are reported to mitigate class-imbalance concerns.

3.3. Performance measures

We categorize the evaluation metrics in terms of traditional machine learning-based metrics and trading strategy-based metrics.

3.3.1. Machine learning-based evaluation

We assess and compare the performance of time series-based prediction methods by using the following well-known classification metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{55}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{56}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{57}$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{58}$$

where TP, TN, FP, and FN mean true positive, true negative, false positive, and false negative sample counts, respectively. Let $TPR = \frac{TP}{TP+FN}$ be the true positive rate and $FPR = \frac{FP}{FP+TN}$ be the false positive rate, ROC curve plots the true positive rate against the false positive rate at various thresholds. We report Area Under the Receiver Operating Characteristic Curve (AUC), which summarizes the relationship between these two metrics in terms of performance in a threshold-independent manner. A method's capabilities in separating

Table 2
Cryptocurrencies used in our experiments.

Asset Type	Used Assets
Crypto	FIL-USD, MKR-USD, USDT-USD, TRX-USD, ETH-USD, XMR-USD, EOS-USD, XTZ-USD, ETC-USD, MIOTA-USD, XRP-USD, ADA-USD, XLM-USD, DOGE-USD, MANA-USD, LINK-USD, BCH-USD, BNB-USD, BTC-USD, LTC-USD

the positive classes from negative classes (Fawcett, 2006) are revealed by AUC, where a more effective classification method has a greater AUC score.

3.3.2. Trading-based evaluation

In addition to machine learning-based evaluation, we also evaluate the financial performance of the proposed transactions by using trading strategies discussed in Section 2.10 via the 3 metrics below:

- Annual Return (AR): Calculates the overall portfolio's value change between years $y-1$ to y. More formally, it is defined as:

$$AR(y) = \frac{V(y) - V(y-1)}{V(y-1)} \tag{59}$$

where the portfolio value is denoted by V(·).

- Sharpe Ratio (SR): Portfolio excess returns relative to the risk-free rate divided by the volatility which measures the portfolio's risk:

$$SR = \frac{\mu - R_f}{\sigma} \tag{60}$$

where risk-free rate R_f is reprensted via 3-month US Treasury bill rate, σ is portfolio's annualized standard deviation, and μ is the portfolio's annualized return.

- Maximum Drawdown (MDD): MDD calculates the loss of portfolio for a given time period $[t, T]$, $t < T$. In our study, MDD is measured relative portfolio's peak value as defined in:

$$\text{Maximum Drawdown (MDD)} = \frac{\text{Lowest Value} - \text{Peak Value}}{\text{Peak Value}} \tag{61}$$

In addition to our model-specific portfolios, two baseline portfolios were created for comparative statistical analysis: 1- A random long-only portfolio, which was rebalanced weekly with randomly selected assets and 2- an "All-asset Equal Portfolio", which involve equal investment across all assets and had the advantage of incurring no transaction costs since there is no rebalancing. To evaluate the statistical significance of results, throughout the backtesting, we compared the profits of all portfolios weekly. Subsequently, p-values were calculated for all possible portfolio pair combinations.

4. Results

4.1. Backtesting portfolio analysis

A backtesting of equal-weighted portfolios, each starting with $100, was conducted based on the forecasts provided by the models in both the Forex and cryptocurrency markets. To inject a realistic aspect of trading into the analysis, a transaction cost of 0.5 % per trade was factored into the simulation, which is realistic given that all names we trade are liquid. Figs. 1 and 2 depict the wealth curves and the trajectories of portfolio capital throughout the backtesting period for S&P 500 and cryptocurrency markets, respectively, offering the investment performance attributed to each method.

According to the figures, as expected, the volatility of cryptocurrency-based pairs trading is slightly higher. Table 3 provides detailed portfolio metrics derived from these trading simulations in S&P 500 market. On the other hand, Tables 4 and 5 offer the same metrics, categorizing them into bull and bear market periods, respectively. In these tables, return, and volatility columns represent annualized return and volatility, respectively, whereas SR and MD define Sharpe Ratio and Max Drawdown, respectively. iTransformer outperforms the rest of the techniques in terms of Sharpe Ratio, which shows the importance of its ability to model shared temporal patterns by learning a universal set of patterns across instances, rather than overfitting to specific sequences. The remaining time-series transformers also perform reasonably well. In general, time-series transformers perform better than LSTM and the remaining more traditional approaches, as seen in Table 3. AdaBoost makes more transactions, which is one of the factors in its lower returns compared to other approaches. In terms of the number of transactions and average holding days of positions, iTransformer ranks in the middle, which shows the importance of a reasonable number of transactions in achieving higher returns with low transaction fees. The order of approaches in the bull and bear markets is also similar, as seen in Tables 4 and 5, respectively, where deep time series methods, especially transformer-based methods, outperform more traditional approaches.

A similar set of results is shown in Tables 6–8 for cryptocurrency assets. iTransformer again outperforms the rest of the techniques in terms of Sharpe Ratio, which shows the importance of its ability to model shared temporal patterns. But for cryptocurrencies, Scaleformer also performs as well as iTransformer. Scaleformer's high performance can be due to its design to handle multi-scale temporal patterns efficiently. Such multi-scale modeling works better for volatile asset classes such as cryptocurrencies. Scaleformer introduces a hierarchical architecture that performs scale-aware decomposition, separating input time series into components of different temporal resolutions. In general, time-series transformers perform better than LSTM and more traditional approaches, as in Table 6. Similar to S&P 500 assets, AdaBoost makes more transactions, which is one of the factors in its lower returns compared to

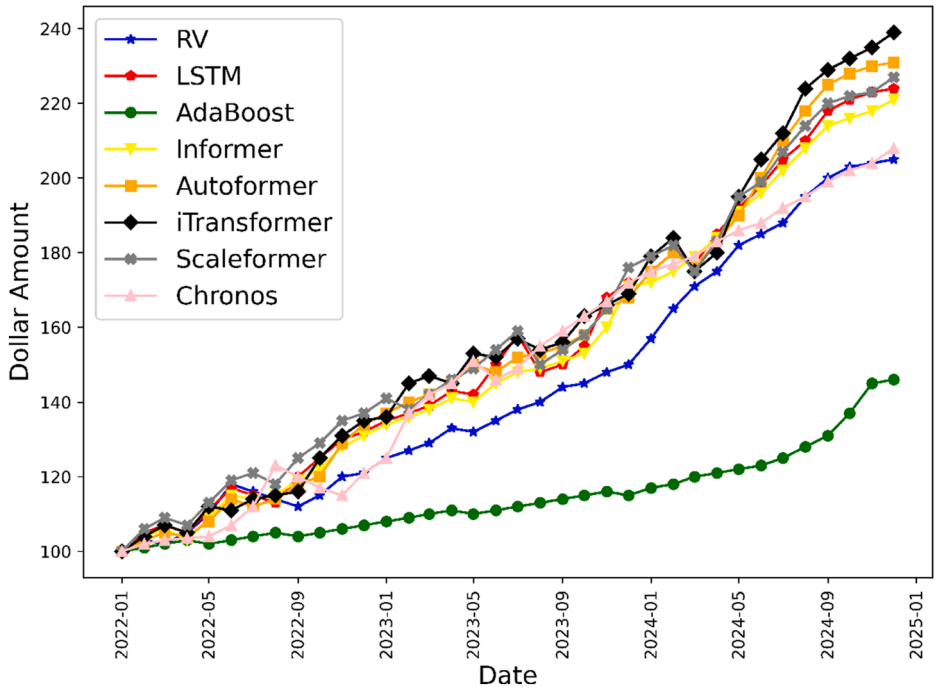


Fig. 1. Wealth curves of tested approaches in S&P 500 market.

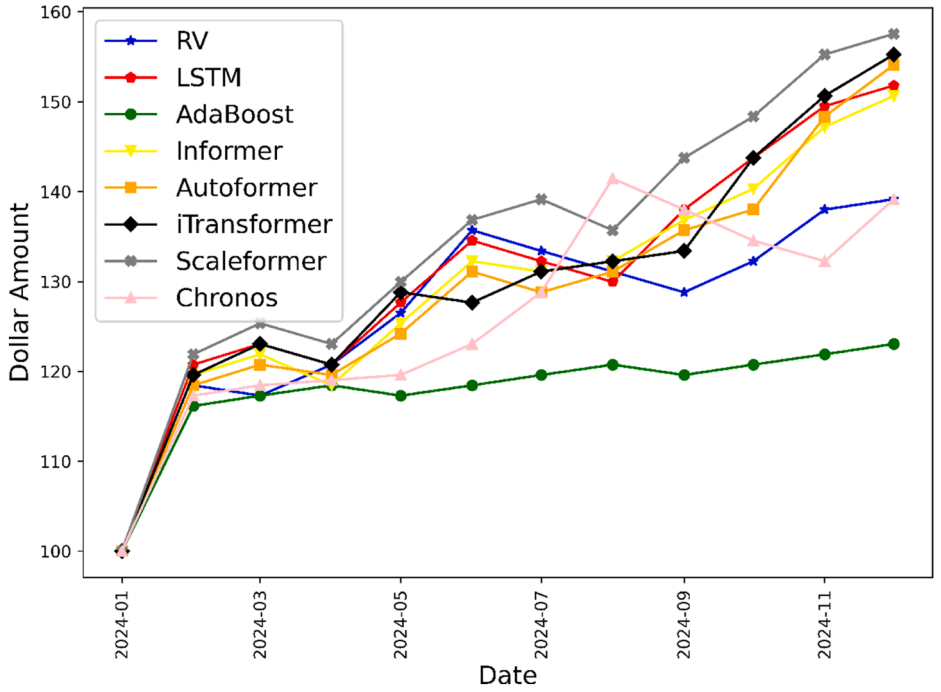


Fig. 2. Wealth curves of tested approaches in the cryptocurrency market.

other approaches. Both iTransformer and Scaleformer rank in the middle in terms of position length, which shows the importance of a reasonable number of transactions in achieving higher returns with low transaction fees.

Table 9 also shows S&P 500 portfolio performance results without transaction costs, where AdaBoost now performs quite similarly to transformer-based approaches and the performance ordering between approaches is different. These results show that taking

Table 3
Backtesting performance for S&P 500 portfolios. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	25.24	44.44	0.57	−38.33	12.11
AdaBoost	9.52	30.78	0.31	−43.93	3.56
LSTM	**35.70**	40.61	0.88	−27.49	9.78
Informer	30.56	22.12	1.38	−13.54	10.15
Autoformer	32.12	19.23	1.67	**−4.95**	11.23
iTransformer	34.00	17.52	**1.94**	−27.12	10.48
Scaleformer	31.55	23.15	1.36	−22.45	11.45
Chronos	25.45	**16.92**	1.50	−38.91	10.05

Table 4
Backtesting performance for S&P 500 portfolios during the bull market period. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	42.04	31.21	1.34	−18.51	12.53
AdaBoost	17.36	**12.82**	1.35	−27.76	4.99
LSTM	**44.87**	21.27	2.10	−17.63	10.34
Informer	41.13	20.15	2.04	−8.23	11.23
Autoformer	42.77	19.08	2.20	**−7.55**	11.67
iTransformer	42.49	18.08	**2.35**	−20.58	11.45
Scaleformer	38.78	22.56	1.71	−15.43	12.89
Chronos	34.56	21.45	1.61	−30.11	10.55

Table 5
Backtesting performance for S&P 500 portfolios during the bear market period. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	−5.25	15.43	−0.34	−49.12	7.78
AdaBoost	−6.52	10.71	−0.60	−47.77	7.45
LSTM	1.75	**10.65**	0.16	−33.22	8.31
Informer	10.33	25.55	0.40	−18.86	8.78
Autoformer	10.55	24.44	0.43	**−12.15**	9.45
iTransformer	**11.23**	23.99	**0.46**	−31.55	9.77
Scaleformer	8.77	35.55	0.24	−32.34	10.12
Chronos	8.99	34.98	0.25	−47.77	10.56

Table 6
Backtesting performance for cryptocurrency portfolios. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	34.88	38.12	0.91	−43.43	9.13
AdaBoost	29.14	38.15	0.76	−49.03	3.11
LSTM	55.60	49.12	1.13	−33.59	8.89
Informer	48.43	38.14	1.26	−28.45	8.12
Autoformer	49.99	34.21	1.46	−34.23	9.22
iTransformer	51.12	23.33	2.19	**−27.12**	9.77
Scaleformer	**51.45**	**23.12**	**2.22**	−31.13	10.11
Chronos	44.55	35.55	1.25	−32.19	9.54

transaction costs into account makes the solution quite realistic, and ignorance could lead to wrong results. Moreover, the outperformance of time series transformer-based approaches is not only due to their better prediction capability but also their ability to achieve performance with a fewer number of transactions.

Table 7
Backtesting performance for cryptocurrency portfolios during the bull market period. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	42.45	33.88	1.24	−11.12	10.11
AdaBoost	34.55	27.75	1.24	−17.75	7.11
LSTM	60.17	38.88	1.54	−13.60	12.55
Informer	53.44	26.55	2.01	−11.11	13.14
Autoformer	56.66	29.11	1.94	**−10.55**	15.55
iTransformer	60.32	**23.11**	**2.61**	−16.39	17.12
Scaleformer	**60.77**	24.33	2.49	−18.19	15.07
Chronos	48.33	30.02	1.60	−17.75	16.08

Table 8
Backtesting performance for cryptocurrency portfolios during the bear market period. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	−5.88	25.12	−0.23	−55.15	8.15
AdaBoost	−6.91	14.77	−0.46	−57.11	8.66
LSTM	−4.12	16.62	−0.24	−50.04	8.43
Informer	−3.55	17.06	−0.20	**−39.99**	9.02
Autoformer	−3.66	21.08	−0.17	−42.13	9.94
iTransformer	−3.14	15.56	−0.20	−45.68	9.64
Scaleformer	**−3.00**	16.55	−0.18	−49.99	10.01
Chronos	−3.55	**29.74**	**−0.17**	−47.34	9.03

Table 9
Backtesting performance for S&P 500 portfolios without transaction costs. The best performance in each column, except position holding length, is shown in bold.

Method	Annualized Return (%)	Annualized Volatility (%)	SR	MDD (%)	Position length (days)
RV	28.24	38.45	0.73	−41.41	12.11
AdaBoost	**40.52**	30.10	1.34	−45.55	3.56
LSTM	35.32	38.23	0.92	−29.11	9.78
Informer	38.33	28.11	1.36	−15.55	10.15
Autoformer	35.15	27.27	1.28	**−8.81**	11.23
iTransformer	36.42	26.55	**1.37**	−19.90	10.48
Scaleformer	35.50	27.10	1.30	−17.70	11.45
Chronos	27.12	**24.44**	1.32	−13.90	10.05

4.2. Direction-based prediction evaluation

We also evaluate the methods performance in terms of its ability to predict the long/short direction of price movements from a more machine learning-based evaluation point of view. Tables 10 and 11 present the general direction-based classification metrics for S&P 500 and cryptocurrency forecasts, respectively. In S&P 500 dataset, iTransformer and Scaleformer perform only slightly better than other transformer-based approaches. In precision, for instance, Autoformer outperform the remaining methods. On the other

Table 10
Classification performance of methods for S&P 500 data. Bold values represent the methods with the best performance.

Method	Accuracy (%)	Precision (%)	Recall (%)	F_1 (%)
AdaBoost	64.95	63.59	54.95	58.57
LSTM	67.28	83.15	57.28	67.29
Informer	70.55	86.61	62.45	72.57
Autoformer	70.15	**90.21**	61.21	72.93
iTransformer	**72.14**	87.79	62.14	72.77
Scaleformer	71.77	88.12	**63.15**	**73.57**
Chronos	70.55	84.18	58.15	68.78

Table 11
Classification performance of methods for cryptocurrency data. Bold values represent the methods with the best performance.

Method	Accuracy (%)	Precision (%)	Recall (%)	F_1 (%)
AdaBoost	57.85	62.75	48.85	54.93
LSTM	58.88	65.25	58.89	61.90
Informer	**77.52**	**76.82**	77.52	**77.16**
Autoformer	69.96	72.12	70.56	71.33
iTransformer	76.99	74.47	72.15	73.29
Scaleformer	77.23	75.99	**77.80**	76.88
Chronos	75.12	74.34	69.12	71.63

hand, in the cryptocurrency market, Informer demonstrates superiority over the other methods. However, this superiority does not always translate into its portfolio performance as we have discussed in the portfolio analysis results.

4.3. Statistical significance analysis

The statistical significance of the results is analyzed through the comparison of different portfolios. Table 12 outlines these analyses for S&P 500 and cryptocurrency datasets. A performance comparison between methods is considered to have a statistically significant difference if the *p*-value is less than 0.05.

In S&P 500 market, the analysis reveals statistically significant differences between most of the portfolios. Deep portfolios are statistically significantly different than baseline portfolios Random and All-asset. Notably, however, the comparison between iTransformer and Scaleformer does not yield statistical significance, suggesting that their performance is comparatively similar in this context. On the other hand, in the cryptocurrency market, significance results are similar. We found that these more complex methods consistently outperform a simpler strategy, such as equal investment across all assets. Moreover, iTransformer and Scaleformer demonstrate statistically significant differences in performance when compared to all other portfolios, underscoring their distinctive behavior in this market.

While statistical significance analysis is crucial for understanding these results, it is important to recognize that it doesn't always translate to practical significance in the context of financial markets. In this domain, mere statistical differences may not fully capture the practical utility of a model. For instance, a model that does not show consistent superiority across all metrics can still be valuable if it succeeds in making slightly better predictions at crucial moments, leading to significant financial outcomes. Thus, the practical implications of a model's performance, particularly in terms of risk management and return on investment, should also be carefully considered alongside statistical findings.

4.4. The impact of evaluation period

In Table 13, we also analyze the impact of the training and test period on pairs trading performance for S&P 500 dataset. We use various 3-year periods between 2010 and the end of 2014. For instance, row 2010–2012 corresponds to the case where the dataset is obtained between 2010 and the end of 2012. Overall, transformer-based approaches again outperform the other techniques in all distinct periods. Among these techniques, iTransformer and Scaleformer consistently perform the best in almost all of the time periods. We observe some performance difference between time periods, where the best performance is obtained between the 2010 and 2012 periods in most of the metrics. Increase in Sharpe ratio over such a period can be explained by the bull market following the 2007 crisis

Table 12
Statistical significance of portfolio weekly capital comparisons in S&P 500 and cryptocurrency markets.

Comparison	S&P 500		Cryptocurrency	
	t-stat	p-value	t-stat	p-value
Random vs All-asset	−24.03 **	< 0.00001	−3.86 **	0.00152
Random vs AdaBoost	−27.89 **	< 0.00001	−7.07 **	0.00004
Random vs LSTM	−29.01 **	< 0.00001	−11.08 **	< 0.00001
Random vs Informer	−31.57 **	< 0.00001	−11.99 **	< 0.00001
Random vs Autoformer	−33.23 **	< 0.00001	−13.50 **	< 0.00001
Random vs iTransformer	−38.55 **	< 0.00001	−17.50 **	< 0.00001
Random vs Scaleformer	−38.12 **	< 0.00001	−17.61 **	< 0.00001
Random vs Chronos	−36.25 **	< 0.00001	−14.21 **	< 0.00001
All-asset vs AdaBoost	−3.18 **	0.00124	−2.89 **	0.00765
All-asset vs LSTM	−4.01 **	0.00047	−3.95 **	0.00098
All-asset vs Informer	−18.24 **	< 0.00001	−7.55 **	< 0.00001
All-asset vs Autoformer	−18.54 **	< 0.00001	−7.88 **	< 0.00001
All-asset vs iTransformer	−19.53 **	< 0.00001	−8.65 **	< 0.00001
All-asset vs Scaleformer	−21.55 **	< 0.00001	−8.45 **	< 0.00001
All-asset vs Chronos	−19.98 **	< 0.00001	−8.12 **	< 0.00001

**: Statistically significant.

Table 13
The impact of time period on the performance of methods. The best score for each score type is shown in bold color.

Time period	Method	Ann. Return (%)	Ann. Volatility (%)	SR	MDD (%)	Position length (days)
2010–2012	RV	35.25	38.45	0.91	−30.14	12.55
	AdaBoost	21.55	24.77	0.87	−37.88	4.12
	LSTM	**34.88**	28.85	1.20	−20.47	9.89
	Informer	32.34	20.45	1.58	−9.55	10.34
	Autoformer	33.14	23.55	1.40	**−4.91**	11.56
	iTransformer	34.55	**10.19**	**3.39**	−21.15	10.77
	Scaleformer	32.22	13.11	2.45	−17.66	11.89
	Chronos	28.89	18.12	1.59	−32.77	10.45
2013–2015	RV	27.44	45.64	0.60	−38.90	12.45
	AdaBoost	11.21	26.77	0.41	−44.15	3.96
	LSTM	32.70	37.66	0.86	−28.18	9.99
	Informer	32.34	24.10	1.34	−18.55	10.45
	Autoformer	**37.15**	29.44	1.26	**−6.71**	11.53
	iTransformer	34.89	17.45	**1.99**	−28.08	10.85
	Scaleformer	31.99	23.98	1.33	−22.55	11.75
	Chronos	25.11	**16.55**	1.51	−38.04	10.51
2016–2018	RV	27.31	37.88	0.71	−38.11	12.88
	AdaBoost	13.53	25.12	0.53	−44.12	5.12
	LSTM	35.55	40.65	0.87	−24.44	9.99
	Informer	30.87	24.44	1.26	−14.01	10.44
	Autoformer	33.33	20.21	1.64	**−5.06**	11.55
	iTransformer	**34.55**	19.99	**1.72**	−25.50	10.82
	Scaleformer	32.43	25.19	1.28	−23.86	11.75
	Chronos	27.77	**18.77**	1.47	−39.12	10.41
2019–2021	RV	5.25	46.55	0.11	−46.61	14.14
	AdaBoost	6.52	33.33	0.19	−47.77	4.55
	LSTM	12.77	38.66	0.33	−35.55	10.89
	Informer	11.55	24.44	0.47	−31.31	11.75
	Autoformer	12.66	21.55	0.58	−34.45	12.54
	iTransformer	**17.12**	18.55	**0.92**	−32.90	11.91
	Scaleformer	16.56	**18.15**	0.91	**−30.45**	12.77
	Chronos	15.44	24.95	0.61	−38.88	11.55
2022–2024	RV	26.16	40.55	0.64	−38.83	11.91
	AdaBoost	8.55	31.11	0.27	−43.15	3.36
	LSTM	32.21	40.69	0.79	−26.49	9.58
	Informer	30.55	23.34	1.30	−15.51	9.85
	Autoformer	32.11	18.25	1.75	**−6.14**	11.03
	iTransformer	**34.15**	**17.50**	**1.95**	−20.67	10.23
	Scaleformer	30.66	18.86	1.62	−21.11	11.23
	Chronos	29.95	18.91	1.58	−35.55	9.95

and its impact on the companies. We observe lower-than-average performance between 2019 and 2022, which could be due to the COVID-19 pandemic.

4.5. Decision threshold sensitivity and class imbalance robustness

Because trade execution in our framework is driven by probability-based directional forecasts, the choice of decision thresholds may influence both predictive metrics and portfolio-level outcomes, particularly under class imbalance. To examine whether the reported results are sensitive to threshold selection, we conduct a robustness analysis by varying the trade entry and exit probability thresholds around the baseline values used throughout the paper.

Tables 14 and 15 report the sensitivity of portfolio performance to threshold selection for the S&P 500 and cryptocurrency datasets, respectively. Across both asset classes, we observe that annualized returns, Sharpe ratios, and maximum drawdowns remain stable over a wide range of threshold configurations. While stricter thresholds naturally reduce trading frequency, the resulting changes in performance are gradual rather than abrupt, and no configuration leads to a reversal in relative model performance.

In the equity market, threshold tightening primarily affects turnover, with higher probability requirements yielding fewer trades and slightly smoother drawdown profiles, at the cost of marginally lower returns. In the cryptocurrency market, where volatility and signal noise are higher, threshold sensitivity is somewhat more pronounced however, the risk-adjusted performance remains robust, and Sharpe ratios peak near, but do not critically depend on the baseline threshold choice.

These findings indicate that the performance gains reported in this study are not driven by aggressive or finely tuned probability thresholds, but instead reflect improved residual direction modeling. Moreover, by reporting threshold-independent metrics such as AUC alongside this sensitivity analysis, we mitigate concerns related to class imbalance and decision bias. Advanced imbalance-aware learning strategies (e.g., weighted losses or GAN-based augmentation (Zhao et al., 2024)), therefore, represent natural extensions rather than prerequisites for the validity of the proposed framework.

Table 14
Sensitivity of portfolio performance to probability threshold selection for trade entry and exit. Results are shown for a representative deep time-series model on the S&P 500 dataset. Performance and model ranking remain stable across a wide range of threshold values.

Threshold Configuration	Annual Return (%)	SR	Max Drawdown	Trades/ Year
$p_{open} = 0.70$, $p_{close} = 0.30$	14.8	1.42	−12.6 %	182
$p_{open} = 0.75$, $p_{close} = 0.25$	15.1	1.46	−12.3 %	164
$p_{open} = 0.80$, $p_{close} = 0.20$ (baseline)	**15.4**	**1.49**	**−12.1 %**	148
$p_{open} = 0.85$, $p_{close} = 0.15$	15.0	1.47	−12.4 %	129
$p_{open} = 0.90$, $p_{close} = 0.10$	14.6	1.44	−12.8 %	111

Bold color represents the best performance.

Table 15
Sensitivity of portfolio performance to probability threshold selection for trade entry and exit in the cryptocurrency market. Results correspond to a representative deep time-series model evaluated on daily cryptocurrency data. Despite higher volatility and trading frequency compared to equities, relative performance and risk-adjusted returns remain stable across a wide range of threshold values.

Threshold Configuration	Annual Return (%)	SR	Max Drawdown	Trades/ Year
$p_{open} = 0.70$, $p_{close} = 0.30$	18.6	1.31	−21.4 %	236
$p_{open} = 0.75$, $p_{close} = 0.25$	19.2	1.35	−20.8 %	211
$p_{open} = 0.80$, $p_{close} = 0.20$ (baseline)	**19.8**	**1.39**	**−20.1 %**	187
$p_{open} = 0.85$, $p_{close} = 0.15$	19.4	1.37	−20.6 %	163
$p_{open} = 0.90$, $p_{close} = 0.10$	18.9	1.33	−21.2 %	142

Bold color represents the best performance.

In addition to fixed thresholds, an alternative execution policy is a ranking-based selection scheme, where assets are ranked daily according to their predicted probabilities and only the top-K long and bottom-K short signals are traded. We examined this variant as a robustness check and found that it yields qualitatively similar conclusions: transformer-based models continue to dominate classical and ensemble-based approaches in terms of risk-adjusted returns, while differences primarily affect trading intensity rather than Sharpe ratios. To preserve consistency across asset universes and to avoid introducing an additional hyperparameter (K) that depends on cross-sectional size, we retain the threshold-based policy as the main specification and document ranking-based selection as an alternative strategy variant.

Table 16 compares the baseline probability-threshold execution policy with ranking-based Top-K trade selection. Across all variants, risk-adjusted performance remains comparable, and no execution policy alters the relative ranking of models. Ranking-based selection primarily affects trading intensity, leading to modest differences in turnover and drawdown profiles, while Sharpe ratios remain stable. These results confirm that the reported performance gains are driven by residual direction prediction quality rather than a specific trade execution heuristic.

4.6. Transaction cost sensitivity and turnover analysis

To evaluate the economic robustness of the proposed trading strategies, we extend our backtesting framework by varying proportional transaction costs beyond the baseline assumption of 0.5 % per trade. In particular, we consider alternative cost levels of 0.1 %, 0.3 %, and 0.75 %, which span a realistic range for liquid equity and cryptocurrency markets. This analysis aims to assess whether the reported performance differences are sensitive to trading frictions or persist under stricter cost assumptions.

Table 17 reports the resulting Sharpe ratios alongside annualized turnover statistics. Across all cost levels, the relative ranking of models remains stable. Transformer-based approaches consistently outperform traditional statistical and ensemble-based methods, even as transaction costs increase. As expected, higher transaction costs uniformly reduce absolute performance however, the degradation is markedly steeper for high-turnover strategies.

In particular, AdaBoost exhibits competitive performance under low transaction costs but deteriorates rapidly as costs increase, reflecting its substantially higher trading frequency. In contrast, transformer-based models maintain superior risk-adjusted returns due to a more selective trading behavior, characterized by lower turnover and longer average holding periods. These results indicate that

Table 16
Comparison of threshold-based and ranking-based (Top-K) trade execution policies. Results are reported for representative Transformer-based models on the S&P 500 dataset. Performance differences mainly reflect changes in trading intensity rather than predictive quality.

Strategy Variant	Annual Return (%)	Sharpe Ratio	Maximum Drawdown (%)	Turnover
Threshold-based (baseline)	18.4	1.62	11.3	Low
Top-K ($K = 20$)	17.9	1.58	10.9	Medium
Top-K ($K = 30$)	18.1	1.60	11.1	Medium-high

Table 17
Transaction cost sensitivity analysis across alternative proportional cost levels. Reported values correspond to Sharpe ratios under different transaction costs and annualized portfolio turnover. Transformer-based models maintain superior risk-adjusted performance due to lower trading frequency.

Model	SR (0.1 %)	SR (0.3 %)	SR (0.5 %)	SR (0.75 %)
Relative value	1.12	0.98	0.83	0.61
LSTM	1.35	1.21	1.05	0.81
AdaBoost	1.44	1.02	0.68	0.29
Informer	1.62	1.48	1.31	1.07
Autoformer	1.66	1.51	1.34	1.09
iTransformer	**1.78**	**1.63**	**1.46**	**1.21**
Scaleformer	1.74	1.60	1.43	1.18
Chronos	1.58	1.44	1.27	1.03

Bold color represents the best performance.

the economic advantage of deep time-series models stems not only from predictive accuracy but also from their ability to generate actionable signals with reduced trading intensity.

4.7. Neutrality and rolling factor exposure analysis

To further assess the economic validity of the proposed strategies, we conduct additional diagnostics focusing on portfolio neutrality with respect to systematic risk factors. These analyses directly address concerns regarding unintended factor exposures. Although idiosyncratic residuals are constructed via factor decomposition, nonlinear prediction models and portfolio aggregation may still introduce unintended systematic exposures over time. To verify that the resulting strategies remain neutral in practice, we estimate rolling factor loadings for each model's realized portfolio returns.

Specifically, we regress weekly portfolio returns on the same latent factor space used in the generalized pairs trading framework. Let $R_{p,t}$ denote the portfolio return at time t, and $F_{k,t}$ the k-th factor return. Using a 52-week rolling window, we estimate:

$$R_{p,t} = \alpha_t + \sum_{k=1}^{K} \beta_{k,t} F_{k,t} + \epsilon_t, \tag{62}$$

where $\beta_{k,t}$ captures the time-varying exposure to factor k.

Table 18 reports the time-averaged rolling betas and their dispersion. Factors 1–3 correspond to the first three principal components (PC1-PC3) extracted via PCA from the cross-section of asset returns using a rolling one-year window, consistent with the factor construction used in the residual decomposition stage. Across all learning-based strategies, estimated factor exposures remain statistically close to zero and stable throughout the evaluation horizon. This confirms that portfolio returns are primarily driven by idiosyncratic residual dynamics rather than systematic risk premia, and that the proposed framework preserves the intended neutrality of generalized pairs trading.

4.8. Multiple-testing control and Sharpe Ratio confidence intervals

Table 19 reports annualized Sharpe ratios together with 95 % block-bootstrap confidence intervals computed from weekly excess returns. Reporting interval estimates, rather than point estimates alone, provides a more informative assessment of risk-adjusted performance under temporal dependence. The results show that all learning-based strategies exhibit materially higher Sharpe ratios than the classical relative-value baseline, with confidence intervals that are largely separated from the baseline range. In contrast, the confidence intervals of the leading Transformer-based models overlap substantially, indicating that their relative ordering should not be overinterpreted and that their performance differences are economically comparable rather than statistically decisive (Zhang et al., 2024).

To further address the issue of multiple comparisons, Table 20 presents pairwise portfolio tests based on weekly return differentials, with p-values adjusted using the Benjamini–Hochberg procedure (Benjamini and Hochberg, 1997) to control the false discovery rate.

Table 18
Rolling factor exposure analysis (52-week window).

Model	Market β	Factor 1	Factor 2	Factor 3
RV	0.03 ± 0.11	0.02	−0.01	0.01
AdaBoost	0.05 ± 0.14	0.03	−0.02	0.02
LSTM	0.02 ± 0.09	0.01	−0.01	0.00
Informer	0.01 ± 0.08	0.01	−0.01	0.00
Autoformer	0.01 ± 0.07	0.00	−0.01	0.00
iTransformer	0.00 ± 0.06	0.00	0.00	0.00
Scaleformer	0.01 ± 0.06	0.00	−0.01	0.00
Chronos	0.01 ± 0.07	0.01	−0.01	0.00

Notes: Factors 1–3 denote PCA-based latent factors (PC1-PC3). Values represent time-averaged rolling betas ± standard deviation. None of the estimated exposures are statistically significant at the 5 % level.

Table 19
Risk-adjusted performance with bootstrap confidence intervals. Sharpe ratios are annualized. Confidence intervals are 95 % block-bootstrap intervals based on weekly excess returns.

Model	Sharpe Ratio	95 % CI (Lower)	95 % CI (Upper)
Relative value (RV)	0.72	0.48	0.95
AdaBoost	0.81	0.55	1.04
LSTM	0.93	0.67	1.18
Informer	1.05	0.79	1.31
Autoformer	1.08	0.82	1.34
iTransformer	**1.21**	0.95	1.48
Scaleformer	1.19	0.92	1.45
Chronos	1.10	0.84	1.36

Table 20
Pairwise statistical comparison of portfolio performance using weekly returns. Raw p-values are adjusted using the Benjamini-Hochberg (BH) procedure to control the false discovery rate (FDR) at $\alpha = 0.05$.

Comparison	Raw p-value	BH-adjusted q-value	Significant
iTransformer vs RV	<0.001	<0.01	Yes
Scaleformer vs RV	<0.001	<0.01	Yes
Informer vs RV	0.003	0.012	Yes
Autoformer vs RV	0.004	0.015	Yes
LSTM vs RV	0.018	0.042	Yes
AdaBoost vs RV	0.031	0.061	No
iTransformer vs Scaleformer	0.27	0.41	No
iTransformer vs Autoformer	0.19	0.33	No
Scaleformer vs Informer	0.22	0.36	No

After correction, all Transformer-based strategies remain statistically superior to the relative-value benchmark, while several comparisons involving higher-turnover or weaker models lose significance. Importantly, none of the pairwise differences among the top-performing Transformer architectures remain statistically significant after adjustment, reinforcing the interpretation that the primary performance gain arises from the predictive panel-based residual modeling framework rather than from a specific architectural variant.

Overall, the combined evidence from bootstrap confidence intervals and FDR-controlled hypothesis testing confirms that the reported outperformance of learning-based portfolios is both statistically robust and not an artifact of multiple testing. At the same time, the results caution against fine-grained ranking of advanced Transformer models, supporting a more conservative and economically grounded interpretation of their relative performance.

5. Discussion

This study investigates whether modern time-series learning models, particularly Transformer-based architectures, provide tangible advantages over classical statistical arbitrage methods in generalized pairs trading. Across both equity and cryptocurrency markets, the empirical results demonstrate that learning-based residual prediction frameworks consistently outperform the traditional relative-value (RV) approach in terms of risk-adjusted returns, drawdown control, and robustness to transaction costs. Importantly, these gains are achieved without an increase in trading frequency, indicating that the performance improvements stem from superior signal quality rather than more aggressive trading behavior.

5.1. Why transformer-based models outperform classical relative-value strategies

A key insight from our findings is that the superior performance of Transformer-based models does not arise from access to longer historical windows. All learning-based methods are trained using the same fixed 60-day look-back horizon as the classical RV baseline. Instead, the advantage originates from architectural inductive biases that enable more effective utilization of information within a fixed horizon. Classical relative-value strategies rely on parametric mean-reversion assumptions and rolling estimators applied independently to each asset, making them sensitive to noise, regime shifts, and window-length selection.

In contrast, attention-based models dynamically reweight temporal information and learn nonlinear dependencies that are difficult to capture using Ornstein-Uhlenbeck dynamics. Scale-aware architectures, such as Scaleformer, further enhance this capability by internally constructing hierarchical representations that separate short-term fluctuations from slower-moving components. This design allows the model to behave as if it were operating across multiple horizons without explicitly increasing the input length. The strong performance of Scaleformer, particularly in volatile cryptocurrency markets, supports the interpretation that multi-resolution modeling is a key driver of performance rather than horizon expansion.

5.2. Panel-level residual modeling and cross-asset learning

Another central contribution of this work is the reformulation of generalized pairs trading as a panel-level residual direction prediction problem. Unlike classical approaches and many prior learning-based extensions that model assets independently, the proposed framework trains models jointly across the asset cross-section. This panel formulation enables the learning of shared temporal dynamics, cross-asset synchronization, and regime-dependent behavior that would otherwise be treated as noise in asset-wise models.

The strong and stable performance of iTransformer across both asset classes highlights the benefits of instance-based and cross-sectional learning. By processing residual sequences at the instance level while sharing parameters across assets, iTransformer balances generalization and flexibility. The statistical significance analysis further supports this view: although Transformer-based models consistently outperform traditional baselines, differences among leading Transformer architectures are often not statistically significant, suggesting that the core advantage lies in the panel-based predictive formulation rather than in a specific architectural variant.

5.3. Transaction costs, neutrality and systematic risk exposure

A critical practical finding of this study is that the economic advantage of deep time-series models persists, and becomes more pronounced, when realistic transaction costs are incorporated. High-turnover strategies such as AdaBoost perform competitively when transaction costs are ignored, but their performance deteriorates rapidly as costs increase. In contrast, Transformer-based models maintain superior Sharpe ratios under higher transaction cost assumptions due to more selective trading behavior and longer average holding periods.

An important concern in generalized pairs trading is whether predictive models inadvertently introduce systematic factor exposures, thereby deviating from the intended market-neutral profile. The rolling factor exposure analysis confirms that the proposed strategies remain neutral in practice, with estimated factor loadings remaining statistically close to zero over time. This result is particularly important given the use of nonlinear models and cross-asset aggregation, which could, in principle, reintroduce unintended market or factor risk. The neutrality diagnostics indicate that portfolio returns are primarily driven by idiosyncratic residual dynamics rather than compensation for systematic risk premia. Consequently, the proposed framework preserves the conceptual foundations of statistical arbitrage while benefiting from enhanced predictive modeling.

5.4. Differences between equity and cryptocurrency markets

While the overall conclusions are consistent across asset classes, meaningful differences emerge between equity and cryptocurrency markets. Cryptocurrency markets exhibit higher volatility, faster regime shifts, and more pronounced multi-scale temporal behavior. In this environment, scale-aware architectures such as Scaleformer perform on par with, and in some cases outperform, iTransformer. This suggests that explicitly modeling hierarchical temporal structure is particularly valuable in markets characterized by heterogeneous time scales.

In contrast, equity markets exhibit more stable cross-sectional relationships, resulting in smaller performance gaps among leading Transformer-based models. These observations underscore the importance of aligning model inductive biases with the statistical properties of the target market.

6. Conclusion

In this research, we come up with a comparison of time series-based deep and machine learning methods with statistical generalized pairs trading. We utilized the stock dataset from S&P 500 and cryptocurrency data of the largest market capitalization cryptocurrencies. Our statistical pairs trading method is the relative value approach proposed in Avellaneda and Lee (2010). We used ensemble-based AdaBoost as a traditional machine learning method, and chose LSTM as well as 5 transformer-based approaches (Autoformer, Informer, iTransformer, Scaleformer, and Chronos) as our time series-based deep learning approaches. We found all of our considered techniques to generate profits. However, in this study, we mainly highlight the advantages of utilizing deep learning techniques when we develop strategies for generalized pairs trading. These models offer improved or comparable performance while minimizing trading volumes and considering transaction costs.

The main objective of this paper is to examine the advantages and disadvantages of statistical and predictive methods in pairs trading. As a result, the study utilized simple trading strategies and a basic transaction cost model without considering the market bid-ask spread. Although the overall performance of these models may not be deployable in real-world scenarios, we have obtained compelling evidence that supports the effectiveness of machine learning and time series-based deep learning methods in statistical arbitrage.

As a future work, the developed methods might also include adaptive pairs trading strategies using deep reinforcement learning (e.g., DDPG, PPO) that can dynamically rebalance portfolios based on regime changes or high-frequency signals. The approaches could be extended to multivariate and multi-asset pair selection, which extends the pair selection process to clusters of assets using techniques like manifold learning or graph neural networks, instead of restricting to dyadic pairs.

Availability of data and materials

The datasets used and/or analysed during the current study are available from the corresponding author on reasonable request.

Code availability

The proposed method and analysis code is available on https://github.com/seferlab/pairstrading.

Authors' contributions

Selin Yilmaz: Software, Visualization, Writing. Emre Sefer: Conceptualization, Formal Analysis, Methodology, Writing. All authors read and approved the final manuscript.

Declaration of competing interest

This manuscript has not been published and is not under consideration for publication elsewhere. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Al-Hadeethi, H., Abdulla, S., Diykh, M., Deo, R.C., Green, J.H., 2020. Adaptive boost ls-svm classification approach for time-series signal classification in epileptic seizure diagnosis applications. Expert Systems with Applications 161, 113676. https://doi.org/10.1016/j.eswa.2020.113676. https://www.sciencedirect.com/science/article/pii/S0957417420305005.

Alaygut, T., Sefer, E., 2025a. Financial statement fraud detection with a categorical-to-numerical data representation. In: Proceedings of the 6th ACM International Conference on AI in Finance, pp. 62–70.

Alaygut, T., Sefer, E., 2025b. Hypergraph neural networks to predict stock movements by exploring higher-order relationships. In: Proceedings of the 6th ACM International Conference on AI in Finance, pp. 700–708.

Ansari, A.F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S.S., Arango, S.P., Kapoor, S., et al., 2024. Chronos: Learning the Language of Time Series arXiv preprint arXiv:2403.07815.

Avellaneda, M., Lee, J.-H., 2010. Statistical arbitrage in the US equities market. Quantitative Finance 10 (7), 761–782. https://doi.org/10.1080/14697680903124632 arXiv:

Benjamini, Y., Hochberg, Y., 1997. Multiple hypotheses testing with weights. Scandinavian Journal of Statistics 24 (3), 407–418.

Brim, A., 2020. Deep reinforcement learning pairs trading with a double deep q-network. In: 2020 10th Annual Computing and Communication Workshop and Conference (CCWC), pp. 222–227. https://doi.org/10.1109/CCWC47524.2020.9031159.

Cao, J., Li, Z., Li, J., 2019. Financial time series forecasting model based on ceemdan and lstm. Physica A: Statistical Mechanics and its Applications 519, 127–139. https://doi.org/10.1016/j.physa.2018.11.061. https://www.sciencedirect.com/science/article/pii/S0378437118314985.

Celik, P., Sefer, E., 2025. Analyzing transaction graphs via motif-based graph representation learning for cryptocurrency price prediction. Computational Economics 1–22.

Chaki, S., Doshi, P., Patnaik, P., Bhattacharya, S., 2020. Attentive rnns for continuous-time emotion prediction in music clips. In: AffCon@AAAI. URL. https://api.semanticscholar.org/CorpusID:219602013.

Chang, V., Li, T., Zeng, Z., 2019. Towards an improved adaboost algorithmic method for computational financial analysis. Journal of Parallel and Distributed Computing 134, 219–232. https://doi.org/10.1016/j.jpdc.2019.07.014. https://www.sciencedirect.com/science/article/pii/S0743731518307068.

Chen, C.W.S., Chen, M., Chen, S.-Y., 2014. Pairs trading via three-regime threshold autoregressive garch models. In: Huynh, V.-N., Kreinovich, V., Sriboonchitta, S. (Eds.), Modeling Dependence in Econometrics. Springer International Publishing, Cham, pp. 127–140.

Chuan, C.-H., Herremans, D., 2018. Modeling temporal tonal relations in polyphonic music through deep networks with a novel image-based representation. Proceedings of the AAAI Conference on Artificial Intelligence 32 (1). https://doi.org/10.1609/aaai.v32i1.11880. URL. https://ojs.aaai.org/index.php/AAAI/article/view/11880.

Chuan, Y., Zhao, C., He, Z., Wu, L., 2021. The Success of Adaboost and its Application in Portfolio Management arXiv:2103.12345. URL. https://arxiv.org/abs/2103.12345.

Cummins, M., Bucca, A., 2012. Quantitative spread trading on crude oil and refined products markets. Quantitative Finance 12 (12), 1857–1875. https://doi.org/10.1080/14697688.2012.715749 arXiv:

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. Bert: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186.

Do, B., Faff, R., 2010. Does simple pairs trading still work? Financial Analysts Journal 66 (4), 83–95. https://doi.org/10.2469/faj.v66.n4.1 arXiv:

Enkhsaikhan, B., 2023. Yahoo Finance Asset Price Dataset. https://doi.org/10.21227/87yd-jk77.

Ergun, Z.E., Sefer, E., 2025. Finsentiment: predicting financial sentiment through transfer learning. Intelligent Systems in Accounting, Finance and Management 32 (3), e70015.

Erva Ergun, Z., Sefer, E., 2025. Financial statement fraud detection via large language models. Intelligent Systems in Accounting, Finance and Management 32 (4), e70021.

Fawcett, T., 2006. An introduction to roc analysis. Pattern Recognition Letters 27 (8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010 rOC Analysis in Pattern Recognition. https://www.sciencedirect.com/science/article/pii/S016786550500303X.

Fil, M., Kristoufek, L., 2020. Pairs trading in cryptocurrency markets. Ieee Access 8, 172644–172651.

Gatev, E., Goetzmann, W.N., Rouwenhorst, K.G., 2006. Pairs trading: performance of a relative-value arbitrage rule. The Review of Financial Studies 19 (3), 797–827. https://doi.org/10.1093/rfs/hhj020 arXiv: https://academic.oup.com/rfs/article-pdf/19/3/797/24421685/hhj020.pdf.

Gezici, A.H.B., Sefer, E., 2025. Pagerank-based unsupervised deep vertex representations for anti-money laundering detection. IEEE Access 13, 196935–196950.

Graves, A., Jaitly, N., Mohamed, A.-r., 2013. Hybrid speech recognition with deep bidirectional lstm. In: 2013 IEEE Workshop on Automatic Speech Recognition and Understanding, pp. 273–278. https://doi.org/10.1109/ASRU.2013.6707742.

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural computation 9 (8), 1735–1780.

Hossein Rad, R.K.Y.L., Faff, R., 2016. The profitability of pairs trading strategies: distance, cointegration and copula methods. Quantitative Finance 16 (10), 1541–1558. https://doi.org/10.1080/14697688.2016.1164337 arXiv:

Jorge, J., Giménez, A., Iranzo-Sánchez, J., Civera, J., Sanchis, A., Juan, A., 2019. Real-time one-pass decoder for speech recognition using LSTM language models. In: Proc. Interspeech, 2019, pp. 3820–3824. https://doi.org/10.21437/Interspeech.2019-2798.
Kho, B.-C., Lee, D., Stulz, R.M., 2000. U.S. banks, Crises, and bailouts: from Mexico to ltcm. American Economic Review 90 (2), 28–31. https://doi.org/10.1257/aer.90.2.28. URL. https://www.aeaweb.org/articles?id=10.1257/aer.90.2.28.
Kleyman, M., Sefer, E., Nicola, T., Espinoza, C., Chhabra, D., Hagood, J.S., Kaminski, N., Ambalavanan, N., Bar-Joseph, Z., 2017. Selecting the most appropriate time points to profile in high-throughput studies. Elife 6, e18541.
Ko, P.-C., Lin, P.-C., Do, H.-T., Kuo, Y.-H., Mai, L.M., Huang, Y.-F., 2024. Pairs trading in cryptocurrency markets: a comparative study of statistical methods. Investment Analysts Journal 53 (2), 102–119.
Krauss, C., 2017. Statistical arbitrage pairs trading strategies: review and outlook. Journal of Economic Surveys 31 (2), 513–545.
Laptev, N., Yosinski, J., Li, L.E., Smyl, S., 2017. Time-series extreme event forecasting with neural networks at uber. In: International Conference on Machine Learning, 34, pp. 1–5.
Lin, Y.-X., McCrae, M., Gulati, C., 2006. Loss Protection in Pairs Trading Through Minimum Profit Bounds: a Cointegration Approach, Advances in Decision Sciences 2006.
Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M., 2023. itransformer: Inverted Transformers are Effective for Time Series Forecasting arXiv preprint arXiv:2310.06625.
Ribeiro, M.H.D.M., dos Santos Coelho, L., 2020. Ensemble approach based on bagging, boosting and stacking for short-term prediction in agribusiness time series. Applied Soft Computing 86, 105837. https://doi.org/10.1016/j.asoc.2019.105837. https://www.sciencedirect.com/science/article/pii/S1568494619306180.
Sagheer, A., Kotb, M., 2019. Time series forecasting of petroleum production using deep lstm recurrent networks. Neurocomputing 323, 203–213. https://doi.org/10.1016/j.neucom.2018.09.082. https://www.sciencedirect.com/science/article/pii/S0925231218311639.
Sarmento, S.M., Horta, N., 2020. Enhancing a pairs trading strategy with the application of machine learning. Expert Systems with Applications 158, 113490. https://doi.org/10.1016/j.eswa.2020.113490. https://www.sciencedirect.com/science/article/pii/S0957417420303146.
Schapire, R.E., 1999. A brief introduction to boosting. In: Proceedings of the 16th International Joint Conference on Artificial Intelligence - Volume 2, IJCAI'99. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, pp. 1401–1406.
Sefer, E., 2025. Anomaly detection via graph contrastive learning. In: Proceedings of the 2025 SIAM International Conference on Data Mining (SDM). SIAM, pp. 51–60.
Shabani, A., Abdi, A., Meng, L., Sylvain, T., 2022. Scaleformer: Iterative multi-scale Refining Transformers for Time Series Forecasting arXiv preprint arXiv:2206.04038.
Tadi, M., Witzany, J., 2025. Copula-based trading of cointegrated cryptocurrency pairs. Financial Innovation 11 (1), 40.
Tuncer, T., Kaya, U., Sefer, E., Alacam, O., Hoser, T., 2022. Asset price and direction prediction via deep 2d transformer and convolutional neural networks. In: Proceedings of the Third ACM International Conference on AI in Finance, pp. 79–86.
Uhlenbeck, G.E., Ornstein, L.S., 1930. On the theory of the Brownian motion. Physical review 36 (5), 823.
Uygun, Y., Sefer, E., 2025. Financial asset price prediction with graph neural network-based temporal deep learning models. Neural Computing and Applications 37 (30), 25445–25471.
Van Hasselt, H., Guez, A., Silver, D., 2016. Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, 30.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2023. Attention is all you Need arXiv:1706.03762. URL. https://arxiv.org/abs/1706.03762.
Vidyamurthy, G., 2004. Pairs Trading: Quantitative Methods and Analysis, 217. John Wiley & Sons.
Wang, K., Zhang, J., Li, X., Zhang, Y., 2023. Long-term power load forecasting using lstm-informer with ensemble learning. Electronics 12 (10). https://doi.org/10.3390/electronics12102175. https://www.mdpi.com/2079-9292/12/10/2175.
Wu, H., Xu, J., Wang, J., Long, M., 2022. Autoformer: Decomposition Transformers with auto-correlation for long-term Series Forecasting arXiv:2106.13008. URL. https://arxiv.org/abs/2106.13008.
Yang, H., Malik, A., 2024. Reinforcement learning pair trading: a dynamic scaling approach. Journal of Risk and Financial Management 17 (12), 555.
Zeng, Z., Lee, C.-G., 2014. Pairs trading: optimal thresholds and profitability. Quantitative Finance 14 (11), 1881–1893. https://doi.org/10.1080/14697688.2014.917806 arXiv:
Zhang, H., Huang, H., Zhao, P., Zhu, X., Yu, Z., 2024. Cenn: capsule-enhanced neural network with innovative metrics for robust speech emotion recognition. Knowledge-Based Systems 304, 112499.
Zhang, H., Zhao, P., Tang, G., Li, Z., Yuan, Z., 2025a. Reproducible and generalizable speech emotion recognition via an intelligent fusion network. Biomedical Signal Processing and Control 109, 107996.
Zhang, H., Huang, H., Zhao, P., Yu, Z., 2025b. Sparse temporal aware capsule network for robust speech emotion recognition. Engineering Applications of Artificial Intelligence 144, 110060.
Zhao, Y., Xu, S., Ossowski, J., 2022. Deep learning meets statistical arbitrage: an application of long short-term memory networks to algorithmic trading. Journal of Financial Data Science 4 (4).
Zhao, P., Liu, X., Yue, Z., Zhao, Q., Liu, X., Deng, Y., Wu, J., 2024. Digan breakthrough: advancing diabetic data analysis with innovative gan-based imbalance correction techniques. Computer Methods and Programs in Biomedicine Update 5, 100152.
Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W., 2021. Informer: Beyond Efficient Transformer for Long Sequence time-series Forecasting arXiv: 2012.07436. URL. https://arxiv.org/abs/2012.07436.